

ranges::copy should say output_iterator somewhere

Document #: P2550R0
Date: 2022-02-16
Project: Programming Language C++
Audience: LEWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>

Contents

- [1 Introduction](#)
 - [1.1 Output concept hierarchy](#)
 - [1.2 Does anything in Ranges use output_iterator?](#)
- [2 Proposal: A Weaker Output Iterator](#)
 - [2.1 Summary of Proposal](#)
- [3 Wording](#)
 - [3.1 concept weak_output_iterator](#)
 - [3.2 concept mergeable](#)
 - [3.3 concept weak_output_range](#)
 - [3.4 Algorithms](#)
 - [3.4.1 ranges::copy](#)
 - [3.4.2 ranges::copy_n](#)
 - [3.4.3 ranges::copy_if](#)
 - [3.4.4 ranges::move](#)
 - [3.4.5 ranges::transform](#)
 - [3.4.6 ranges::replace_copy](#)
 - [3.4.7 ranges::fill](#)
 - [3.4.8 ranges::generate](#)
 - [3.4.9 ranges::remove_copy](#)
 - [3.4.10 ranges::unique_copy](#)
 - [3.4.11 ranges::reverse_copy](#)
 - [3.4.12 ranges::rotate_copy](#)
 - [3.4.13 ranges::partition_copy](#)
 - [3.4.14 ranges::merge](#)
 - [3.4.15 ranges::set_union](#)
 - [3.4.16 ranges::set_intersection](#)
 - [3.4.17 ranges::set_difference](#)
 - [3.4.18 ranges::set_symmetric_difference](#)
 - [3.4.19 ranges::iota](#)
- [4 Acknowledgements](#)
- [5 References](#)

§ 1 Introduction

In the beginning, we had:

```
template<class InputIterator, class OutputIterator>
constexpr OutputIterator copy(InputIterator first, InputIterator last,
                             OutputIterator result);
```

And then, in C++20, we added Concepts and Ranges - which came with a whole library of `concepts` for the standard library, in particular for iterators and ranges. This included a concept `input_iterator` and `output_iterator`. We also added new ranged versions of all the algorithms, constrained using these concepts.

The new overloads look like this:

```
template<input_iterator I, sentinel_for<I> S, weakly_incremtable O>
    requires indirectly_copyable<I, O>
    constexpr ranges::copy_result<I, O> ranges::copy(I first, S last, O result);
template<input_range R, weakly_incremtable O>
    requires indirectly_copyable<iterator_t<R>, O>
    constexpr ranges::copy_result<borrowed_iterator_t<R>, O> ranges::copy(R&& r, O result);
```

The crux of this paper is that `std::copy` takes an `OutputIterator` (even if the name of this parameter does nothing), but `std::ranges::copy` does not have anything named `output_iterator` in this constraints at all. That just seems wrong. `Output iterator` is a thing that users understand, but “weakly incremtable” and “indirectly copyable,” while reasonable names for the functionality they require, are not particularly well known and are not really useful when they show up in diagnostics.

We should do better here.

§ 1.1 Output concept hierarchy

Let me start with what all the relevant `concepts` actually are:

```
template<class I>
    concept weakly_incremtable =
        movable<I> &&
        requires(I i) {
            typename iter_difference_t<I>;
            requires is_signed_integer_like<iter_difference_t<I>>;
            { ++i } -> same_as<I&>; // not required to be equality-preserving
            i++; // not required to be equality-preserving
        };

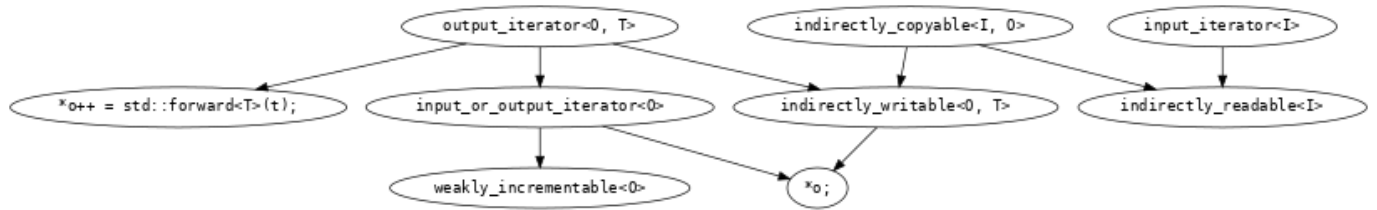
template<class I>
    concept input_or_output_iterator =
        requires(I i) {
            { *i } -> can_reference;
        } &&
        weakly_incremtable<I>;

template<class Out, class T>
    concept indirectly_writable =
        requires(Out&& o, T&& t) {
            *o = std::forward<T>(t); // not required to be equality-preserving
            *std::forward<Out>(o) = std::forward<T>(t); // not required to be equality-preserving
            const_cast<const iter_reference_t<Out>&&>(*o) =
                std::forward<T>(t); // not required to be equality-preserving
            const_cast<const iter_reference_t<Out>&&>(*std::forward<Out>(o)) =
                std::forward<T>(t); // not required to be equality-preserving
        };

template<class In, class Out>
    concept indirectly_copyable =
        indirectly_readable<In> &&
        indirectly_writable<Out, iter_reference_t<In>>;

template<class I, class T>
    concept output_iterator =
        input_or_output_iterator<I> &&
        indirectly_writable<I, T> &&
        requires(I i, T&& t) {
            *i++ = std::forward<T>(t); // not required to be equality-preserving
        };
```

Or, in graph form:



And let me present two possible specifications for `ranges::copy`.

In C++20	Hypothetical
<pre>template <input_iterator I, sentinel_for<I> S, weakly_incrementable O> requires indirectly_copyable<I, O> constexpr ranges::copy_result<I, O> ranges::copy(I first, S last, O result);</pre>	<pre>template <input_iterator I, sentinel_for<I> S, output_iterator<iter_reference_t<I>> O> constexpr ranges::copy_result<I, O> ranges::copy(I first, S last, O result);</pre>

Now, the one on the right actually says `output_iterator`, which I think is extremely valuable. But are the requirements any different?

In C++20, we require:

- `O` is `weakly_incrementable`
- `indirectly_copyable<I, O>`, which means `I` is `indirectly_readable` (which is already required by `input_iterator`) and `O` is `indirectly_writable<iter_reference_t<I>>`

The hypothetical version requires `output_iterator<iter_reference_t<I>>`, which breaks down into:

- `input_or_output_iterator`
 - dereferencable (already required by `indirectly_writable`, since you have to have `*o = expr; work`)
 - `weakly_incrementable` (explicitly required in C++20)
- `indirectly_writable` (explicitly required in C++20)
- `*i++ = t;` (*not* required in C++20)

Basically: the two formulations have identical requirements *except* that today's specification of `std::ranges::copy` does not require `*out++ = t;` to work, while my hypothetical one does. Generally speaking, this is a good thing. It may be syntactically nice to write `*out++ = t;` instead of `*out = t; ++out;`, but it's not actually necessary to solve any problems. This makes `ranges::copy` more usable, but it means that our most output-y of output algorithms doesn't use `output_iterator`.

§ 1.2 Does anything in Ranges use `output_iterator`?

I thought it'd be useful to go through everything in `<algorithm>` that uses an output iterator (if not an `output_iterator`) and catalogue all the kinds of constraints we have on them. There are many different approaches (in the below, `X` just denotes some type, `O` is our output iterator, `I` is the corresponding input iterator):

- `weakly_incrementable<O> && indirectly_copyable<I, O>`: `ranges::copy`, `ranges::copy_n`, `ranges::copy_if`, `ranges::remove_copy`, `ranges::remove_copy_if`, `ranges::unique_copy` (although this one has a disjunction that might include other constraints), `ranges::reverse_copy`, `ranges::rotate_copy`, `ranges::partition_copy`
- `weakly_incrementable<O> && indirectly_movable<I, O>`: `ranges::move`, `ranges::move_if`
- `weakly_incrementable<O> && indirectly_writable<O, X>`: `ranges::transform`
- `output_iterator<O, X> && indirectly_copyable<I, O>`: `ranges::replace_copy`, `ranges::replace_copy_if`

- `output_iterator<O, X>: ranges::fill, ranges::fill_n`
- `input_or_output_iterator<O> && indirectly_writable<O, X>: ranges::generate (see below), ranges::generate_n`
- `weakly_incrementable<O> && mergeable<I1, I2, O, X, X>` (technically `mergeable`'s requirements on `O` are just `indirectly_copyable`, but putting it separately for completeness): `ranges::merge, ranges::set_union, ranges::set_intersection, ranges::set_difference, ranges::set_symmetric_difference`

Put differently, there are only 4 algorithms that use `output_iterator`: `ranges::replace_copy`, `ranges::replace_copy_if`, `ranges::fill`, and `ranges::fill_n`.

There are, separately, 2 algorithms that use `output_range<R>` (which requires its iterator to be an `output_iterator`): `ranges::fill` and `ranges::generate`. While `ranges::fill` has the same requirements on its iterator/sentinel and range overloads, `ranges::generate` does not. The consequence of this is, for instance:

```
auto some_generator() -> std::generator<int&>;
auto some_func() -> int;

void f() {
    auto g = some_generator();
    std::ranges::generate(g, some_func);           // error
    std::ranges::generate(g.begin(), g.end(), some_func); // ok
}
```

Since [\[P2502R0\]](#)'s generator (like all other input-only ranges in the standard library right now) has a postfix `operator++` that returns `void`, this makes `*out++` ill-formed, which means that `iterator_t<generator<int&>>` is not an `output_iterator<int&>` which means that `generator<int&>` is not an `output_range<int&>`. That makes the range overload fail. But `iterator_t<generator<int&>>` is `weakly_incrementable` and `indirectly_writable<int&>`, which are all the requirements of the iterator/sentinel overload, so this... works? The inconsistency is a problem.

Now, `indirectly_copyable<I, O>` is really two constraints put together. It requires that `I` is an `input_iterator` and that `O` is `indirectly_writable<iter_reference_t<I>>`. This constraint is by far the most common formulation for output ranges. But it's a bit redundant, since all of these algorithms already separately require `input_iterator<I>`. The only new requirement that `indirectly_copyable` brings in is the `indirectly_writable` one. What I mean is that instead of:

```
template<input_iterator I, sentinel_for<I> S, weakly_incrementable O>
    requires indirectly_copyable<I, O>
    constexpr ranges::copy_result<I, O> ranges::copy(I first, S last, O result);
```

we could get the same exact same requirements (no more, no less) by instead writing:

```
template<input_iterator I, sentinel_for<I> S, weakly_incrementable O>
    requires indirectly_writable<O, iter_reference_t<I>>
    constexpr ranges::copy_result<I, O> ranges::copy(I first, S last, O result);
```

The same idea could hold for the algorithms requiring `indirectly_movable` (replaced with a different kind of `indirectly_writable` constraint).

§ 2 Proposal: A Weaker Output Iterator

We can't remove the `*out++ = r;` requirement from `output_iterator`. It's 2022, surely somebody has written some C++20 code by now, and might rely on that part of the `concept`. Similarly, we cannot add the `*out++ = r;` requirement to all the algorithms which take an output iterator, since likewise somebody could have written C++20 code that passes in a type into these algorithms that meets every requirement but that one, and this added constraint would break their code.

However, the current state of affairs isn't great. Algorithm requirements are inconsistent and aren't written using terms that the algorithms have historically used, which users are familiar with.

If this were 2019 or 2020, I would suggest that we either drop the `*out++ = r;` requirement from `output_iterator` or strengthen all the algorithms to require `output_iterator`. But it's 2022, and we can clearly do neither. Consequently, this paper does not propose anything that would change the behavior of any valid C++20 code.

Instead, the problem this paper seeks to solve is to unify the requirements that all the output algorithms use. We cannot unify around the stronger concept, so instead we can introduce a new, weaker output iterator concept:

```
template<class I, class T>
concept weak_output_iterator =
    input_or_output_iterator<I> &&
    indirectly_writable<I, T>;

template<class I, class T>
concept output_iterator =
    weak_output_iterator<I, T> &&
    requires(I i, T&& t) {
        *i++ = std::forward<T>(t);           // not required to be equality-preserving
    };
```

With such a concept, we can go through all the algorithms and respecify them to just use it. For example, `ranges::copy` becomes:

In C++20	Proposed
<pre>template <input_iterator I, sentinel_for<I> S, weakly_incrementable O> requires indirectly_copyable<I, O> constexpr ranges::copy_result<I, O> ranges::copy(I first, S last, O result);</pre>	<pre>template <input_iterator I, sentinel_for<I> S, weak_output_iterator<iter_reference_t<I>> O> constexpr ranges::copy_result<I, O> ranges::copy(I first, S last, O result);</pre>

Unlike my hypothetical spelling earlier, these two now have identical requirements.

§ 2.1 Summary of Proposal

This proposal introduces:

- A new concept `weak_output_iterator`, that `output_iterator` adds the `*out++ = r;` requirement on top of.
- A new concept `weak_output_range`, which requires `weak_output_iterator`. `output_range` is re-specified to refine `weak_output_range`.
- Modifying the mergeable concept to use `weak_output_iterator`. This does not change the requirements of this concept in any way.
- All output algorithms now require `weak_output_iterator`
 - In most cases, that's re-specifying `weakly_incrementable` and `indirectly_writable` (no requirements change, just better name)
 - In some cases, that's *weakening* the requirement on those algorithms that require `output_iterator` (all currently valid code is still valid)

As a result, all the output algorithms will have the same constraints (including different overloads of the same algorithm), and all those constraints will have `output_iterator` in them somewhere (even if it's `weak_output_iterator`).

§ 3 Wording

§ 3.1 concept weak_output_iterator

Change 23.2 [iterator.synopsis]:

```

namespace std {
    // ...

    // [iterator.concept.output], concept output_iterator
+ template<class I, class T>
+ concept weak_output_iterator = see below;

    template<class I, class T>
        concept output_iterator = see below;

    // ...
}

```

Change 23.3.4.10 [\[iterator.concept.output\]](#) [Editor's note: The semantic effects are specific to `output_iterator`, not `weak_output_iterator`]:

- 1 The `weak_output_iterator` and `output_iterator` ~~concept defines~~ `concepts define` requirements for a type that can be used to write values (from the requirement for `indirectly_writable` (`[iterator.concept.writable]`)) and which can be both pre- and post-incremented. [Note 1: Output iterators are not required to model `equality_comparable`. — end note]

```

+ template<class I, class T>
+ concept weak_output_iterator =
+     input_or_output_iterator<I> &&
+     indirectly_writable<I, T>;

    template<class I, class T>
        concept output_iterator =
-         input_or_output_iterator<I> &&
-         indirectly_writable<I, T> &&
+         weak_output_iterator<I, T> &&
        requires(I i, T&& t) {
            *i++ = std::forward<T>(t);           // not required to be equality-preserving
        };

```

- 2 Let `E` be an expression such that `decltype((E))` is `T`, and let `i` be a dereferenceable object of type `I`. `I` and `T` model `output_iterator<I, T>` only if `*i++ = E`; has effects equivalent to: `*i = E`; `++i`;

§ 3.2 concept mergeable

Change 23.3.7.7 [\[alg.req.mergeable\]](#):

- 1 The `mergeable` concept specifies the requirements of algorithms that merge sorted sequences into an output sequence by copying elements.

```

template<class I1, class I2, class Out, class R = ranges::less,
        class P1 = identity, class P2 = identity>
concept mergeable =
    input_iterator<I1> &&
    input_iterator<I2> &&
-    weakly_incrementable<Out> &&
-    indirectly_copyable<I1, Out> &&
-    indirectly_copyable<I2, Out> &&
+    weak_output_iterator<Out, iter_reference_t<I1>> &&
+    weak_output_iterator<Out, iter_reference_t<I2>> &&
    indirect_strict_weak_order<R, projected<I1, P1>, projected<I2, P2>>;

```

§ 3.3 concept weak_output_range

Change 24.2 [\[ranges.syn\]](#):

```

namespace std::ranges {
    // ...
}

```

```

    // [range.refinements], other range refinements
+ template<class R, class T>
+   concept weak_output_range = see below;

    template<class R, class T>
        concept output_range = see below;

    // ...
}

```

Change 24.4.5 [\[range.refinements\]](#):

- 1 The `output_range` `weak_output_range` concept specifies requirements of a range type for which `ranges::begin` returns a model of `output_iterator` `weak_output_iterator` (`[iterator.concept.output]`). `output_range`, `input_range`, `forward_range`, `bidirectional_range`, and `random_access_range` are defined similarly.

```

+ template<class R, class T>
+   concept weak_output_range =
+       range<R> && weak_output_iterator<iterator_t<R>, T>;

    template<class R, class T>
        concept output_range =
-       range<R> && output_iterator<iterator_t<R>, T>;
+       weak_output_range<R> && output_iterator<iterator_t<R>, T>;

    template<class T>
        concept input_range =
            range<T> && input_iterator<iterator_t<T>>;

    template<class T>
        concept forward_range =
            input_range<T> && forward_iterator<iterator_t<T>>;

    template<class T>
        concept bidirectional_range =
            forward_range<T> && bidirectional_iterator<iterator_t<T>>;

    template<class T>
        concept random_access_range =
            bidirectional_range<T> && random_access_iterator<iterator_t<T>>;

```

§ 3.4 Algorithms

Change all the constraints to use `weak_output_iterator` or `weak_output_range` in the algorithms. The wording diff here only includes the synopsis, the same changes need to be made in these algorithms' corresponding definition too. These are all in 25.4 [\[algorithm.syn\]](#), broken up by algorithm for convenience:

§ 3.4.1 `ranges::copy`

```

namespace std::ranges {
    template<class I, class O>
        using copy_result = in_out_result<I, O>;

-   template<input_iterator I, sentinel_for<I> S, weakly_incrementable O>
-   requires indirectly_copyable<I, O>
+   template<input_iterator I, sentinel_for<I> S, weak_output_iterator<iter_reference_t<I>> O>
+   constexpr copy_result<I, O>
        copy(I first, S last, O result);

-   template<input_range R, weakly_incrementable O>
-   requires indirectly_copyable<iterator_t<R>, O>
+   template<input_range R, weak_output_iterator<range_reference_t<R>> O>
+   constexpr copy_result<borrowed_iterator_t<R>, O>

```

```

    copy(R&& r, 0 result);
}

```

§ 3.4.2 ranges::copy_n

```

namespace std::ranges {
    template<class I, class O>
        using copy_n_result = in_out_result<I, O>;

-   template<input_iterator I, weakly_increamentable O>
-   requires indirectly_copyable<I, O>
+   template<input_iterator I, weak_output_iterator<iter_reference_t<I>> O>
    constexpr copy_n_result<I, O>
        copy_n(I first, iter_difference_t<I> n, O result);
}

```

§ 3.4.3 ranges::copy_if

```

namespace std::ranges {
    template<class I, class O>
        using copy_if_result = in_out_result<I, O>;

-   template<input_iterator I, sentinel_for<I> S, weakly_increamentable O,
+   template<input_iterator I, sentinel_for<I> S, weak_output_iterator<iter_reference_t<I>> O,
        class Proj = identity,
        indirect_unary_predicate<projected<I, Proj>> Pred>
-   requires indirectly_copyable<I, O>
    constexpr copy_if_result<I, O>
        copy_if(I first, S last, O result, Pred pred, Proj proj = {});

-   template<input_range R, weakly_increamentable O,
+   template<input_range R, weak_output_iterator<range_reference_t<R>> O,
        class Proj = identity,
        indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
-   requires indirectly_copyable<iterator_t<R>, O>
    constexpr copy_if_result<borrowed_iterator_t<R>, O>
        copy_if(R&& r, O result, Pred pred, Proj proj = {});
}

```

§ 3.4.4 ranges::move

```

namespace std::ranges {
    template<class I, class O>
        using move_result = in_out_result<I, O>;

-   template<input_iterator I, sentinel_for<I> S, weakly_increamentable O>
-   requires indirectly_movable<I, O>
+   template<input_iterator I, sentinel_for<I> S,
        weak_output_iterator<iter_rvalue_reference_t<I>> O>
    constexpr move_result<I, O>
        move(I first, S last, O result);

-   template<input_range R, weakly_increamentable O>
-   requires indirectly_movable<iterator_t<R>, O>
+   template<input_range, weak_output_iterator<range_rvalue_reference_t<R>> O>
    constexpr move_result<borrowed_iterator_t<R>, O>
        move(R&& r, O result);
}

```

§ 3.4.5 ranges::transform

[Drafting note: Here, I'm using class `O` and having a trailing `requires weak_output_iterator<O, T>` because the relevant type `T` here is based on `Proj`, which is declared after `O`.]


```

namespace std::ranges {
    template<class I, class O>
        using unary_transform_result = in_out_result<I, O>;

-   template<input_iterator I, sentinel_for<I> S, weakly_incrementable O,
+   template<input_iterator I, sentinel_for<I> S, class O,
        copy_constructible F, class Proj = identity>
-   requires indirectly_writable<O, indirect_result_t<F&, projected<I, Proj>>>
+   requires weak_output_iterator<O, indirect_result_t<F&, projected<I, Proj>>>
    constexpr unary_transform_result<I, O>
        transform(I first1, S last1, O result, F op, Proj proj = {});

-   template<input_range R, weakly_incrementable O, copy_constructible F,
+   template<input_range R, class O, copy_constructible F,
        class Proj = identity>
-   requires indirectly_writable<O, indirect_result_t<F&, projected<iterator_t<R>, Proj>>>
+   requires weak_output_iterator<O, indirect_result_t<F&, projected<iterator_t<R>, Proj>>>
    constexpr unary_transform_result<borrowed_iterator_t<R>, O>
        transform(R&& r, O result, F op, Proj proj = {});

    template<class I1, class I2, class O>
        using binary_transform_result = in_in_out_result<I1, I2, O>;

    template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2, sentinel_for<I2> S2,
-   weakly_incrementable O, copy_constructible F, class Proj1 = identity,
+   class O, copy_constructible F, class Proj1 = identity,
        class Proj2 = identity>
-   requires indirectly_writable<O, indirect_result_t<F&, projected<I1, Proj1>,
+   requires weak_output_iterator<O, indirect_result_t<F&, projected<I1, Proj1>,
        projected<I2, Proj2>>>
    constexpr binary_transform_result<I1, I2, O>
        transform(I1 first1, S1 last1, I2 first2, S2 last2, O result,
            F binary_op, Proj1 proj1 = {}, Proj2 proj2 = {});

-   template<input_range R1, input_range R2, weakly_incrementable O,
+   template<input_range R1, input_range R2, class O,
        copy_constructible F, class Proj1 = identity, class Proj2 = identity>
-   requires indirectly_writable<O, indirect_result_t<F&, projected<iterator_t<R1>, Proj1>,
+   requires weak_output_iterator<O, indirect_result_t<F&, projected<iterator_t<R1>, Proj1>,
        projected<iterator_t<R2>, Proj2>>>
    constexpr binary_transform_result<borrowed_iterator_t<R1>, borrowed_iterator_t<R2>, O>
        transform(R1&& r1, R2&& r2, O result,
            F binary_op, Proj1 proj1 = {}, Proj2 proj2 = {});
}

```

§ 3.4.6 ranges::replace_copy

[Drafting note: This is one of the algorithms that had already required output_iterator, its requirements are being weakened.]

```

namespace std::ranges {
    template<class I, class O>
        using replace_copy_result = in_out_result<I, O>;

    template<input_iterator I, sentinel_for<I> S, class T1, class T2,
-   output_iterator<const T2&> O,
+   weak_output_iterator<const T2&> O,
        class Proj = identity>
-   requires indirectly_copyable<I, O> &&
+   requires weak_output_iterator<O, iter_reference_t<I>>
        indirect_binary_predicate<ranges::equal_to, projected<I, Proj>, const T1*>
    constexpr replace_copy_result<I, O>
        replace_copy(I first, S last, O result, const T1& old_value, const T2& new_value,
            Proj proj = {});

    template<input_range R, class T1, class T2,
-   output_iterator<const T2&> O,

```

```

+         weak_output_iterator<const T2&> 0,
            class Proj = identity>
-         requires indirectly_copyable<iterator_t<R>, 0> &&
+         requires weak_output_iterator<0, range_reference_t<R>> &&
            indirect_binary_predicate<ranges::equal_to,
                                    projected<iterator_t<R>, Proj>, const T1*>
            constexpr replace_copy_result<borrowed_iterator_t<R>, 0>
                replace_copy(R&& r, 0 result, const T1& old_value, const T2& new_value,
                            Proj proj = {});

            template<class I, class O>
                using replace_copy_if_result = in_out_result<I, O>;

            template<input_iterator I, sentinel_for<I> S, class T,
-                 output_iterator<const T&> 0,
+                 weak_output_iterator<const T&> 0,
                    class Proj = identity, indirect_unary_predicate<projected<I, Proj>> Pred>
-                 requires indirectly_copyable<I, 0>
+                 requires weak_output_iterator<0, iter_reference_t<I>>
                    constexpr replace_copy_if_result<I, 0>
                        replace_copy_if(I first, S last, 0 result, Pred pred, const T& new_value,
                                        Proj proj = {});
            template<input_range R, class T,
-                 output_iterator<const T&> 0,
+                 weak_output_iterator<const T&> 0,
                    class Proj = identity,
                    indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
-                 requires indirectly_copyable<iterator_t<R>, 0>
+                 requires weak_output_iterator<0, range_reference_t<R>>
                    constexpr replace_copy_if_result<borrowed_iterator_t<R>, 0>
                        replace_copy_if(R&& r, 0 result, Pred pred, const T& new_value,
                                        Proj proj = {});
}

```

§ 3.4.7 ranges::fill

[Drafting note: This was the one, consistent algorithm that required both output_iterator and output_range. Now it (still consistently) requires weak_output_iterator and weak_output_range.]

```

namespace std::ranges {
    template<class T,
-         output_iterator<const T&> 0,
+         weak_output_iterator<const T&> 0,
        sentinel_for<0> S>
        constexpr 0 fill(0 first, S last, const T& value);
    template<class T,
-         output_range<const T&> R>
+         weak_output_range<const T&> R>
        constexpr borrowed_iterator_t<R> fill(R&& r, const T& value);
    template<class T,
-         output_iterator<const T&> 0>
+         weak_output_iterator<const T&> 0>
        constexpr 0 fill_n(0 first, iter_difference_t<0> n, const T& value);
}

```

§ 3.4.8 ranges::generate

[Drafting note: This was the inconsistent algorithm, which now becomes consistent]

```

namespace std::ranges {
-     template<input_or_output_iterator 0,
+     template<class 0,
        sentinel_for<0> S, copy_constructible F>
        requires invocable<F&> &&
-         indirectly_writable<0, invoke_result_t<F&>>

```

```

+         weak_output_iterator<O, invoke_result_t<F&>>
            constexpr O generate(O first, S last, F gen);

        template<class R, copy_constructible F>
            requires invocable<F&> &&
-         output_range<R, invoke_result_t<F&>>
+         weak_output_range<R, invoke_result_t<F&>>
            constexpr borrowed_iterator_t<R> generate(R&& r, F gen);

-     template<input_or_output_iterator O,
+     template<class O,
            copy_constructible F>
            requires invocable<F&> &&
-         indirectly_writable<O, invoke_result_t<F&>>
+         weak_output_iterator<O, invoke_result_t<F&>>
            constexpr O generate_n(O first, iter_difference_t<O> n, F gen);
}

```

§ 3.4.9 ranges::remove_copy

```

namespace std::ranges {
    template<class I, class O>
        using remove_copy_result = in_out_result<I, O>;

    template<input_iterator I, sentinel_for<I> S,
-         weakly_incrementable O,
+         weak_output_iterator<iter_reference_t<I>> O,
            class T,
            class Proj = identity>
-         requires indirectly_copyable<I, O> &&
+         requires
            indirect_binary_predicate<ranges::equal_to, projected<I, Proj>, const T*>
            constexpr remove_copy_result<I, O>
                remove_copy(I first, S last, O result, const T& value, Proj proj = {});

    template<input_range R,
-         weakly_incrementable O,
+         weak_output_iterator<range_reference_t<R>> O,
            class T, class Proj = identity>
-         requires indirectly_copyable<iterator_t<R>, O> &&
+         requires
            indirect_binary_predicate<ranges::equal_to,
                                    projected<iterator_t<R>, Proj>, const T*>
            constexpr remove_copy_result<borrowed_iterator_t<R>, O>
                remove_copy(R&& r, O result, const T& value, Proj proj = {});

    template<class I, class O>
        using remove_copy_if_result = in_out_result<I, O>;

    template<input_iterator I, sentinel_for<I> S,
-         weakly_incrementable O,
+         weak_output_iterator<iter_reference_t<I>> O,
            class Proj = identity, indirect_unary_predicate<projected<I, Proj>> Pred>
-         requires indirectly_copyable<I, O>
            constexpr remove_copy_if_result<I, O>
                remove_copy_if(I first, S last, O result, Pred pred, Proj proj = {});

    template<input_range R,
-         weakly_incrementable O,
+         weak_output_iterator<range_reference_t<R>> O,
            class Proj = identity,
            indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
-         requires indirectly_copyable<iterator_t<R>, O>
            constexpr remove_copy_if_result<borrowed_iterator_t<R>, O>
                remove_copy_if(R&& r, O result, Pred pred, Proj proj = {});
}

```

§ 3.4.10 `ranges::unique_copy`

[Drafting note: The constraints here have a disjunction, but `indirectly_copyable<I, O>` is always required, which is the `weak_output_iterator` constraint.]

```
namespace std::ranges {
    template<class I, class O>
        using unique_copy_result = in_out_result<I, O>;

    template<input_iterator I, sentinel_for<I> S,
-         weakly_incrementable O,
+         weak_output_iterator<iter_reference_t<I>> O,
        class Proj = identity,
        indirect_equivalence_relation<projected<I, Proj>> C = ranges::equal_to>
-     requires indirectly_copyable<I, O> &&
+     requires
        (forward_iterator<I> ||
         (input_iterator<O> && same_as<iter_value_t<I>, iter_value_t<O>>) ||
         indirectly_copyable_storable<I, O>)
        constexpr unique_copy_result<I, O>
            unique_copy(I first, S last, O result, C comp = {}, Proj proj = {});

    template<input_range R,
-         weakly_incrementable O,
+         weak_output_iterator<range_reference_t<R>> O,
        class Proj = identity,
        indirect_equivalence_relation<projected<iterator_t<R>, Proj>> C =
            ranges::equal_to>
-     requires indirectly_copyable<iterator_t<R>, O> &&
+     requires
        (forward_iterator<iterator_t<R>> ||
         (input_iterator<O> && same_as<range_value_t<R>, iter_value_t<O>>) ||
         indirectly_copyable_storable<iterator_t<R>, O>)
        constexpr unique_copy_result<borrowed_iterator_t<R>, O>
            unique_copy(R&& r, O result, C comp = {}, Proj proj = {});
}
```

§ 3.4.11 `ranges::reverse_copy`

```
namespace std::ranges {
    template<class I, class O>
        using reverse_copy_result = in_out_result<I, O>;

    template<bidirectional_iterator I, sentinel_for<I> S,
-         weakly_incrementable O>
+         weak_output_iterator<iter_reference_t<I>> O>
-     requires indirectly_copyable<I, O>
        constexpr reverse_copy_result<I, O>
            reverse_copy(I first, S last, O result);

    template<bidirectional_range R,
-         weakly_incrementable O>
+         weak_output_iterator<range_reference_t<R>> O>
-     requires indirectly_copyable<iterator_t<R>, O>
        constexpr reverse_copy_result<borrowed_iterator_t<R>, O>
            reverse_copy(R&& r, O result);
}
```

§ 3.4.12 `ranges::rotate_copy`

```
namespace std::ranges {
    template<class I, class O>
        using rotate_copy_result = in_out_result<I, O>;

    template<forward_iterator I, sentinel_for<I> S,
-         weakly_incrementable O>
```

```

+         weak_output_iterator<iter_reference_t<I>> 0>
-         requires indirectly_copyable<I, 0>
            constexpr rotate_copy_result<I, 0>
                rotate_copy(I first, I middle, S last, 0 result);

-         template<forward_range R, weakly_incrementable 0>
+         template<forward_range R, weak_output_iterator<range_reference_t<R>> 0>
-         requires indirectly_copyable<iterator_t<R>, 0>
            constexpr rotate_copy_result<borrowed_iterator_t<R>, 0>
                rotate_copy(R&& r, iterator_t<R> middle, 0 result);
}

```

§ 3.4.13 ranges::partition_copy

```

namespace std::ranges {
    template<class I, class O1, class O2>
        using partition_copy_result = in_out_out_result<I, O1, O2>;

    template<input_iterator I, sentinel_for<I> S,
-         weakly_incrementable O1, weakly_incrementable O2,
+         weak_output_iterator<iter_reference_t<I>> O1,
+         weak_output_iterator<iter_reference_t<I>> O2,
            class Proj = identity, indirect_unary_predicate<projected<I, Proj>> Pred>
-         requires indirectly_copyable<I, O1> && indirectly_copyable<I, O2>
            constexpr partition_copy_result<I, O1, O2>
                partition_copy(I first, S last, O1 out_true, O2 out_false, Pred pred,
                    Proj proj = {});

    template<input_range R,
-         weakly_incrementable O1, weakly_incrementable O2,
+         weak_output_iterator<range_reference_t<R>> O1,
+         weak_output_iterator<range_reference_t<R>> O2,
            class Proj = identity,
            indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
-         requires indirectly_copyable<iterator_t<R>, O1> &&
-         indirectly_copyable<iterator_t<R>, O2>
            constexpr partition_copy_result<borrowed_iterator_t<R>, O1, O2>
                partition_copy(R&& r, O1 out_true, O2 out_false, Pred pred, Proj proj = {});
}

```

§ 3.4.14 ranges::merge

[Drafting note: mergeable now requires `weak_output_iterator<0, iter_reference_t<I1>>` and `weak_output_iterator<0, iter_reference_t<I2>>` (for the two input iterators, `I1`, and `I2`). The extra `weakly_incrementable<0>` here doesn't really add anything, but keeping it around would make the merging algorithms the only ones that require `weakly_incrementable`, which is differently consistent. So I'm suggesting we change to `class`.]

```

namespace std::ranges {
    template<class I1, class I2, class O>
        using merge_result = in_in_out_result<I1, I2, O>;

    template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2, sentinel_for<I2> S2,
-         weakly_incrementable O, class Comp = ranges::less, class Proj1 = identity,
+         class O, class Comp = ranges::less, class Proj1 = identity,
            class Proj2 = identity>
        requires mergeable<I1, I2, O, Comp, Proj1, Proj2>
        constexpr merge_result<I1, I2, O>
            merge(I1 first1, S1 last1, I2 first2, S2 last2, O result,
                Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});

-         template<input_range R1, input_range R2, weakly_incrementable O, class Comp = ranges::less,
+         template<input_range R1, input_range R2, class O, class Comp = ranges::less,
            class Proj1 = identity, class Proj2 = identity>
            requires mergeable<iterator_t<R1>, iterator_t<R2>, O, Comp, Proj1, Proj2>
            constexpr merge_result<borrowed_iterator_t<R1>, borrowed_iterator_t<R2>, O>

```

```

        merge(R1&& r1, R2&& r2, 0 result,
              Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});
    }

```

§ 3.4.15 ranges::set_union

[Drafting note: Same idea as ranges::merge, here and for the other set algorithms]

```

namespace std::ranges {
    template<class I1, class I2, class O>
        using set_union_result = in_in_out_result<I1, I2, O>;

    template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2, sentinel_for<I2> S2,
-         weakly_incrementable O, class Comp = ranges::less,
+         class O, class Comp = ranges::less,
        class Proj1 = identity, class Proj2 = identity>
        requires mergeable<I1, I2, O, Comp, Proj1, Proj2>
        constexpr set_union_result<I1, I2, O>
            set_union(I1 first1, S1 last1, I2 first2, S2 last2, O result, Comp comp = {},
                    Proj1 proj1 = {}, Proj2 proj2 = {});

-     template<input_range R1, input_range R2, weakly_incrementable O,
+     template<input_range R1, input_range R2, class O,
        class Comp = ranges::less, class Proj1 = identity, class Proj2 = identity>
        requires mergeable<iterator_t<R1>, iterator_t<R2>, O, Comp, Proj1, Proj2>
        constexpr set_union_result<borrowed_iterator_t<R1>, borrowed_iterator_t<R2>, O>
            set_union(R1&& r1, R2&& r2, O result, Comp comp = {},
                    Proj1 proj1 = {}, Proj2 proj2 = {});
}

```

§ 3.4.16 ranges::set_intersection

```

namespace std::ranges {
    template<class I1, class I2, class O>
        using set_intersection_result = in_in_out_result<I1, I2, O>;

    template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2, sentinel_for<I2> S2,
-         weakly_incrementable O, class Comp = ranges::less,
+         class O, class Comp = ranges::less,
        class Proj1 = identity, class Proj2 = identity>
        requires mergeable<I1, I2, O, Comp, Proj1, Proj2>
        constexpr set_intersection_result<I1, I2, O>
            set_intersection(I1 first1, S1 last1, I2 first2, S2 last2, O result,
                           Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});

-     template<input_range R1, input_range R2, weakly_incrementable O,
+     template<input_range R1, input_range R2, class O,
        class Comp = ranges::less, class Proj1 = identity, class Proj2 = identity>
        requires mergeable<iterator_t<R1>, iterator_t<R2>, O, Comp, Proj1, Proj2>
        constexpr set_intersection_result<borrowed_iterator_t<R1>, borrowed_iterator_t<R2>, O>
            set_intersection(R1&& r1, R2&& r2, O result,
                           Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});
}

```

§ 3.4.17 ranges::set_difference

```

namespace std::ranges {
    template<class I, class O>
        using set_difference_result = in_out_result<I, O>;

    template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2, sentinel_for<I2> S2,
-         weakly_incrementable O, class Comp = ranges::less,
+         class O, class Comp = ranges::less,
        class Proj1 = identity, class Proj2 = identity>
        requires mergeable<I1, I2, O, Comp, Proj1, Proj2>
        constexpr set_difference_result<I1, O>

```

```

        set_difference(I1 first1, S1 last1, I2 first2, S2 last2, 0 result,
                      Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});

- template<input_range R1, input_range R2, weakly_incrementable O,
+ template<input_range R1, input_range R2, class O,
    class Comp = ranges::less, class Proj1 = identity, class Proj2 = identity>
requires mergeable<iterator_t<R1>, iterator_t<R2>, O, Comp, Proj1, Proj2>
constexpr set_difference_result<borrowed_iterator_t<R1>, O>
    set_difference(R1&& r1, R2&& r2, O result,
                  Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});
}

```

§ 3.4.18 ranges::set_symmetric_difference

```

namespace std::ranges {
    template<class I1, class I2, class O>
        using set_symmetric_difference_result = in_in_out_result<I1, I2, O>;

    template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2, sentinel_for<I2> S2,
-         weakly_incrementable O, class Comp = ranges::less,
+         class O, class Comp = ranges::less,
        class Proj1 = identity, class Proj2 = identity>
requires mergeable<I1, I2, O, Comp, Proj1, Proj2>
constexpr set_symmetric_difference_result<I1, I2, O>
    set_symmetric_difference(I1 first1, S1 last1, I2 first2, S2 last2, O result,
                            Comp comp = {}, Proj1 proj1 = {},
                            Proj2 proj2 = {});

- template<input_range R1, input_range R2, weakly_incrementable O,
+ template<input_range R1, input_range R2, class O,
    class Comp = ranges::less, class Proj1 = identity, class Proj2 = identity>
requires mergeable<iterator_t<R1>, iterator_t<R2>, O, Comp, Proj1, Proj2>
constexpr set_symmetric_difference_result<borrowed_iterator_t<R1>,
                                         borrowed_iterator_t<R2>, O>
    set_symmetric_difference(R1&& r1, R2&& r2, O result, Comp comp = {},
                            Proj1 proj1 = {}, Proj2 proj2 = {});
}

```

§ 3.4.19 ranges::iota

[Drafting note: This one is in `<numeric>`, just adopted by way of [\[P2440R1\]](#)]:

```

namespace std::ranges {
    template<class O, class T>
        using iota_result = out_value_result<O, T>;

- template<input_or_output_iterator O, sentinel_for<O> S, weakly_incrementable T>
- requires indirectly_writable<O, const T>
+ template<weakly_incrementable T, weak_output_iterator<const T> O, sentinel_for<O> S>
    constexpr iota_result<O, T> iota(O first, S last, T value);

- template<weakly_incrementable T, output_range<const T> R>
+ template<weakly_incrementable T, weak_output_range<const T> R>
    constexpr iota_result<borrowed_iterator_t<R>, T> iota(R&& r, T value);
}

```

§ 4 Acknowledgements

Thanks to Tim Song for all the help.

§ 5 References

[P2440R1] Tim Song. 2021-12-06. ranges::iota, ranges::shift_left, and ranges::shift_right.

<https://wg21.link/p2440r1>

[P2502R0] Casey Carter. 2021-12-13. std::generator: Synchronous Coroutine Generator for Ranges.

<https://wg21.link/p2502r0>